```
  else
    render action :edit
  end
end
```

And here's the test with the complete task that specifies email behavior:

display/04/spec/requests/task_requests_spec.rb

```
it "sends email when task is completed" do
  patch(task_path(id: task.id), params: {task: {completed: true}})
  expect(ActionMailer::Base.deliveries.size).to eq(1)
  email = ActionMailer::Base.deliveries.first
  expect(email.subject).to eq("A task has been completed")
  expect(email.to).to eq(["monitor@tasks.com"])
  expect(email.body.to_s).to match(/Learn how to test mailers/)
end
```

Again you're simulating the request to the update method, this time with a pseudo-attribute, completed, which you can assume indicates a check box of some kind. Note that you're not testing the actual behavior of completing the test here; that would normally be a model or workflow concern.

Instead, you're testing the mailer. You can verify that one email has been sent by checking the count of ActionMailer::Base.deliveries and then looking at the email object, ActionMailer::Base.deliveries.first,[4] and querying it for its subject and the list of addresses it's going to. You can also check that the body contains the task's title. (Generally the accessors have the names you would expect.)

The passing controller logic uses the completed param to trigger whether the task is updated and the mailer is sent—usually I'd create a workflow object here, but I'll spare you looking at a separate file and keep the code in the controller:

display/04/app/controllers/tasks_controller.rb

```
def update
  @task = Task.find(params[:id])
  completed = params[:task][:completed] == "true" && !@task.complete?
  params[:task][:completed_at] = Time.current if completed
  if @task.update_attributes(task_params)
    TaskMailer.task_completed_email(@task).deliver if completed
    redirect_to @task, notice: "project was successfully updated"
  else
    render action :edit
  end
end
```

4. http://guides.rubyonrails.org/action_mailer_basics.html